

AI Technologies

Cours et méthodes, cours 1 à 3 et lab 1

BMEVIAUMSMA001, AUT, automne 2026. Rédigé le 2026-09-28.

À retenir	
Test écrit (ZH)	un seul pendant le semestre ; date À VÉRIFIER
Seuil ZH	plus de 40 % (condition de signature)
Rattrapage ZH	une possibilité en période de rattrapage, date À VÉRIFIER
Labs	6 labs en ligne, asynchrones ; au moins 4 validés
Validation d'un lab	40 % des points (message du 2026-09-23) ; le cours 1 dit « note 2 au moins »
Rendu d'un lab	pull request GitHub, notée ; deadlines fermes, aucun lab rattrapable
Projet final	devoir maison à rendre avant la fin du semestre ; sujet et date À VÉRIFIER
Note finale	pondération ZH, labs, projet À VÉRIFIER

Source : fiche CLAUDE.md de la matière ; C1, p. 1

Piège

Le cours 1 (p. 1) parle de « 4 out of the 7 » labs. La fiche officielle, le message du 2026-09-23 et le site des labs en listent six. Retenir six : le chiffre du support est probablement une coquille (**À VÉRIFIER** auprès de l'équipe si le doute persiste).

Abréviations des sources (numéros de page : ceux du PDF).

- C1 01-advanced-ai-and-python-foundations.pdf, 19 p.
- C2 02-advanced-python-numpy-pandas.pdf, 38 p.
- C3 03-tensors-and-framework-internals.pdf, 18 p.
- L1 lab01-enonce.md, copie du site des labs
- NB1 à NB3 notebooks 01.ipynb, 02.ipynb, 03.ipynb

Statut du contenu. Tout ce qui vient des supports cite sa page. Les compléments (formules de softmax ou d'entropie croisée, complexités, comptage de

paramètres) sont marqués (*rappel général*). Les sorties de code montrées ont été produites en exécutant le code (Python 3.14, NumPy 2.5) ; PyTorch, TensorFlow et Pandas n'étant pas installés sur la machine de rédaction, aucune sortie de ces bibliothèques n'est donnée hors de ce que le support affiche lui-même. L'incertain est marqué **À VÉRIFIER**.

Périmètre. Les supports disponibles couvrent les semaines 1 à 3 du programme. Les thèmes des semaines 4 à 14 (outils d'assistance au code, IA générative, RAG, agents, FastAPI, MLOps) ne sont pas encore distribués et ne sont pas traités ici.

Table des matières

1	Organisation du cours et programme	4
2	IA symbolique et IA connexionniste	5
3	Formulation probabiliste de l'apprentissage	6
4	CPython : bytecode et machine virtuelle	7
5	Modèle mémoire : des étiquettes sur des objets	9
6	Types primitifs et typage	10
7	Contrôle de flux et protocole d'itération	11
8	Structures de données et compréhensions	12
9	Environnements virtuels et dépendances	13
10	La spirale en Python pur et ses limites	14
11	Fonctions, classes et visibilité	15
12	Cycle de vie d'un objet : <code>__new__</code> et <code>__init__</code>	17
13	Héritage multiple, MRO et linéarisation C3	18
14	Héritage coopératif : <code>*args</code> et <code>**kwargs</code>	19
15	Méthodes magiques et fermetures	21
16	Décorateurs et métaprogrammation	22
17	Google Colab et architecture Jupyter	24
18	ndarray : buffer, métadonnées et strides	25
19	Vues et copies : slicing et fancy indexing	27
20	SIMD et vectorisation	28

21	Broadcasting	29
22	Fonctions NumPy pour un pipeline de ML	30
23	Pandas : moteur en colonnes pour les données	32
24	Alignement par index et passage aux tenseurs	33
25	Spirale vectorisée, GIL et choix des outils	34
26	Matériel : CPU, GPU, SIMT et tenseur	35
27	PyTorch : philosophie et allocateur à cache	37
28	Autograd : différentiation automatique	38
29	nn.Module, couches et activations	39
30	Boucle d'entraînement PyTorch	41
31	Mode eager, define-by-run et torch.compile	43
32	TensorFlow et Keras	44
33	Comparer PyTorch et TensorFlow	46
34	JAX, XLA et TPU	47
35	La spirale classée par PyTorch	48
36	Préparer les labs	50
37	Méthode d'examen	52
38	Aide-mémoire	54

1. Organisation du cours et programme

Source : C1, p. 1 à 3 ; fiche CLAUDE.md

Le cours 1 annonce l'approche : ce n'est pas un cours d'initiation à la programmation. Il explique comment Python implémente les concepts en C (CPython), en quoi il diffère de C++ et Java, et pourquoi ces choix en font le langage de fait de l'IA malgré sa lenteur ; il passe ensuite des systèmes à règles aux paradigmes probabilistes (C1, p. 2 et 3).

Sem.	Thème (fiche officielle)
1	Introduction to the AI and ML Ecosystem
2	Python Fundamentals and Numerical Computing
3	AI Frameworks : PyTorch, TensorFlow, JAX
4, 5	AI-Based Software Development I et II (assistance au code, agentic coding)
6, 7	Generative AI : usages, Transformers, embeddings, HuggingFace
8, 9	LLM locaux, bases vectorielles, RAG ; services et multimodal
10, 11	Agents, MCP, Agent2Agent ; IA dans le cloud
12, 13	Déploiement FastAPI et Docker ; MLOps
14	The Future of AI

Les trois supports reçus portent les titres « Module 1 : The Foundations of Modern AI and Python Internals », « Module 2 : Advanced Python Architectures and Infrastructure » et « Module 3 : Deep Learning Foundations, Architecture, Tensors, and Framework Internals ». Ils sont rédigés comme un polycopié (texte suivi, code, schémas), pas comme des diapositives. Le cours 1 donne aussi des liens vers deux matières recommandées (VIAUBXAV087, VIAUBXAV081) et vers les « Project Laboratories » de l'AUT (C1, p. 1).

À retenir

Le fil rouge des trois cours est un même exemple : le jeu de données en spirale. Il est généré en Python pur (C1), réécrit en NumPy vectorisé (C2), puis classé par un réseau PyTorch (C3). Savoir expliquer ce qui change d'une version à l'autre résume le périmètre.

2. IA symbolique et IA connexionniste

Source : C1, p. 3 et 4

La recherche en IA s’est longtemps partagée entre deux écoles. Le cours insiste sur cette distinction pour expliquer pourquoi l’apprentissage profond (connexionnisme) l’a emporté sur les tâches de perception.

	Symbolique (GOFAI)	Connexionniste (deep learning)
Époque	années 1950 à 1990	années 1980 à aujourd’hui, explosion après 2012
Idée	manipuler des symboles selon des règles formelles explicites	l’intelligence émerge de l’interaction parallèle de nombreuses unités simples (neurones)
Vue informatique	problème de recherche et de logique	problème d’optimisation et d’approximation de fonction
Représentation	graphes de connaissances, ontologies, règles si-alors (Prolog, systèmes experts)	vecteurs et matrices (tenseurs) de flottants
Raisonnement	déduction, résolution, parcours de graphe (recherche A^*)	logique encodée implicitement dans les poids θ
Matériel	machines Lisp (listes chaînées, ramasse-miettes)	GPU, architectures massivement SIMD

Pourquoi le symbolique a échoué. Il est impossible d’encoder à la main la variabilité infinie du monde réel : on ne peut pas écrire assez de if pour reconnaître un chat sous tous les éclairages et dans toutes les poses. Le schéma de la p. 4 appelle ce mur le « Knowledge Bottleneck ».

Schéma p. 4. À gauche, « Era 1 » : un arbre de recherche d’un état initial à un chemin, des portes logiques et le symbole A^* . À droite, « Era 2 » : un réseau en couches (entrée, couches cachées, sortie) avec des poids $W_1, W_2, \dots$ et un grand θ , légendé « Success : implicit encoding in weights ».

Piège

Le support écrit « GPUs (Massive SIMD) » pour le connexionnisme (C1, p. 4), puis le cours 3 précise que les GPU suivent le modèle SIMT (C3, p. 1 et 2). À l'examen, pour une question sur les GPU, employer SIMT ; SIMD désigne les registres vectoriels des CPU (concept 20).

3. Formulation probabiliste de l'apprentissage

Source : C1, p. 4 et 5 ; résumé p. 19

En IA moderne on ne raisonne plus en règles mais en distributions de probabilité : on suppose que les données viennent d'un processus statistique inconnu, que l'on cherche à approcher.

Supervisé : modélisation discriminative, $P(Y | X)$

On apprend la probabilité conditionnelle d'une étiquette Y sachant une entrée X : « étant donnée cette image X , quelle est la probabilité qu'elle soit de la classe Chat (Y) ? ». On cherche des paramètres θ d'une fonction f tels que

$$f(X; \theta) \approx P(Y | X).$$

Exemples du support : classification d'images (X tenseur de pixels $224 \times 224 \times 3$, Y vecteur de probabilités sur 1000 classes) ; détection de spam (X texte du mail, $Y \in \{0, 1\}$) ; diagnostic médical (X analyses sanguines, Y score de risque).

Non supervisé : modélisation générative, $P(X)$

Pas d'étiquettes, seulement des données X . On apprend la distribution des données elles-mêmes, $P(X)$: « à quoi ressemblent des données réalistes ? ». On minimise la distance entre la distribution produite par le modèle et celle des données réelles. Exemples :

- **détection d'anomalies** : on apprend $P(X)$ sur du trafic réseau normal ; si un nouveau paquet x_{new} a $P(x_{new})$ très faible (par exemple 10^{-9}), c'est probablement une attaque ;
- **IA générative** (Stable Diffusion, Midjourney) : connaissant $P(X)$ sur les

images, on échantillonne de nouvelles images synthétiques réalistes ;

- **clustering** : regrouper des clients selon leurs achats sans connaître les groupes à l'avance.

Classer un cas

« Prédire si une transaction est frauduleuse à partir de 10 000 transactions étiquetées » : supervisé, $P(Y | X)$ avec $Y \in \{0, 1\}$. « Repérer des connexions inhabituelles sans aucun exemple d'attaque » : non supervisé, $P(X)$ appris sur le normal, seuil sur $P(x_{new})$.

Piège

Ne pas inverser les notations : supervisé = $P(Y | X)$ (conditionnelle), non supervisé = $P(X)$ (distribution des données). Le résumé du cours (p. 19) associe le passage au connexionnisme aux « modèles probabilistes $P(Y | X)$ ».

4. CPython : bytecode et machine virtuelle

Source : C1, p. 5 à 7

Python est souvent dit « interprété », ce qui simplifie : c'est un langage **interprété par bytecode**, comme Java, avec une machine virtuelle à pile.

Chaîne d'exécution (schéma p. 6). Source `.py` → compilateur (parseur) → arbre syntaxique abstrait (AST) → bytecode (`.pyc`, mis en cache dans `__pycache__` pour accélérer les imports suivants) → PVM. Le bytecode est un jeu d'instructions bas niveau et indépendant de la plateforme.

La PVM (Python Virtual Machine) n'est pas un matériel : c'est une immense boucle infinie écrite en C (`ceval.c` dans les sources de CPython). Elle lit une instruction, exécute la fonction C correspondante (par exemple `PyNumber_Add`), passe à la suivante. L'illustration p. 7 montre ce motif : `for (; ;)` contenant un `switch` (opcode).

Pourquoi `a + b` est plus lent qu'en C++

En C++, `a + b` sur des entiers devient une instruction CPU (`ADD EAX, EBX`). En Python, c'est une instruction de bytecode d'addition, et la PVM

doit :

1. vérifier le type de a ;
2. vérifier le type de b ;
3. chercher la méthode `__add__` de a ;
4. appeler (dispatch) ;
5. créer un nouvel objet pour le résultat.

Ce coût de « dispatch dynamique » se paie à chaque instruction.

Vérification avec le module `dis` (Python 3.14.6)

```
import dis
def f(a, b):
    return a + b
dis.dis(f)
```

Sortie obtenue (extrait) : `LOAD_FAST BORROW LOAD_FAST BORROW 1` (a, b), puis `BINARY_OP 0 (+)`, puis `RETURN_VALUE`.

Piège

Le support nomme l'instruction `BINARY_ADD`. C'était son nom jusqu'à Python 3.10 ; depuis 3.11 les opérateurs binaires passent par `BINARY_OP` (sortie ci-dessus). Pour le ZH, citer le nom du support en sachant expliquer le mécanisme. Le support dit aussi la VM « stack-based and row-major » : seul « à pile » a un sens pour une VM ; l'origine de « row-major » dans cette phrase est **À VÉRIFIER**.

5. Modèle mémoire : des étiquettes sur des objets

Source : C1, p. 7 et 8 ; résumé p. 19 ; devoir p. 19

En C++, la mémoire est un ensemble de boîtes. En Python, c'est une « soupe » d'objets sur le tas, et les variables sont des étiquettes (post-it) collées dessus. Le cours parle d'« appel par référence d'objet ».

Instruction	C++ (boîte)	Python (étiquette)
a = 5	une boîte a contient 5	un objet 5 (PyObject) sur le tas, étiquette « a » collée dessus
b = a	nouvelle boîte b, 5 est copié	étiquette « b » collée sur le <i>même</i> objet : un objet, deux étiquettes
a = 10	la boîte a contient 10, b vaut 5	nouvel objet 10, l'étiquette « a » y est déplacée ; « b » reste sur 5

Objets mutables. Avec `list_b = list_a` puis `list_a.append(4)`, on suit l'étiquette jusqu'à l'objet et on le modifie en place : `list_b` affiche `[1, 2, 3, 4]`. Le schéma p. 8 oppose « The C++ Box » (pile, accès direct, copie de valeur) et « The Python Tag » (tas, référence indirecte, objet partagé).

Gestion de la mémoire. Le résumé (p. 19) indique que la mémoire est gérée par comptage de références plus un ramasse-miettes (GC). Le cours 2 (p. 37) relie le GIL à la sûreté de ce comptage (concept 25).

Devoir 2 du cours 1, exécuté

```
a = [1, 2, 3]; b = a; b[0] = 99
print(a, a is b)           # [99, 2, 3] True
a = [1, 2, 3]; b = a; b = 99
print(a)                   # [1, 2, 3]
x = 5; y = x; y = 10
print(x, y)                # 5 10
```

Piège

L'énoncé du devoir écrit « Modify b = 99 » en attendant que a change. Une réaffectation `b = 99` déplace seulement l'étiquette b : a ne change

pas. Seule une modification en place (`b[0] = 99`, `b.append(...)`) se voit par `a`. L'intention exacte de l'énoncé est **À VÉRIFIER** ; savoir expliquer les deux cas.

6. Types primitifs et typage

Source : C1, p. 8 et 9

Python est à **typage dynamique** (pas de déclaration de type, le type est celui de l'objet à l'exécution) mais **fortement typé** : pas de conversion implicite comme `1 + "1"` en JavaScript. En Python, cette expression lève `TypeError: unsupported operand type(s) for +: 'int' and 'str'` (sortie obtenue).

Type	Ce que dit le support
int	précision arbitraire, pas de long ni short ; grandit tant que la RAM le permet, pas de dépassement. Littéraux hexadécimaux <code>0xFF</code>
float	équivalent du double C++ (64 bits). Notation scientifique <code>1.2e-4</code>
bool	sous-classe de <code>int</code> : <code>True</code> vaut 1, <code>False</code> vaut 0
str	séquence immuable de caractères Unicode. f-strings (Python 3.6+) : préfixe <code>f</code> , expressions entre accolades, compilées en instructions efficaces. Chaînes brutes <code>r"..."</code> qui ignorent les échappements (regex, chemins)
None	équivalent du pointeur nul, objet singleton ; toujours comparer avec <code>is</code> , jamais avec <code>==</code>

Sorties obtenues

```
print(isinstance(True, int), True + True)    # True 2
print(2**100)                                # 1267650600228229401496703205376
version = 3.10
print(f"v{version:.1f}", version)            # v3.1 3.1
```

Piège

L'exemple du support écrit `version = 3.10` : c'est un float, qui vaut 3.1. Un numéro de version se stocke en chaîne ("3.10"). Autre piège : `is` teste l'identité (même objet), `==` l'égalité de valeur.

7. Contrôle de flux et protocole d'itération

Source : C1, p. 10 et 11

L'indentation remplace les accolades et elle est obligatoire. Les opérateurs sont des mots anglais.

Opération	C++	Python
ET, OU, NON	<code>&&, , !</code>	<code>and, or, not</code>
Identité	comparaison de pointeurs	<code>is</code>
Appartenance	<code>std::find</code>	<code>in</code>

Les comparaisons se chaînent : `0 < x < 10` équivaut à `x > 0 and x < 10`. Les branches s'écrivent `if`, `elif` (et non `else if`), `else`.

La boucle `for` est un « pour chaque »

Elle ne manipule pas d'indice ; elle demande un itérateur à l'objet :

1. elle appelle `iter(objet)`, qui cherche la méthode spéciale `__iter__` de la classe ;
2. elle appelle `next(itérateur)` jusqu'à l'exception `StopIteration` ;
3. à défaut de `__iter__`, elle essaie `__getitem__` avec les indices 0, 1, 2, etc.

Pour un indice, utiliser `enumerate()`, un générateur qui produit des tuples (0, 'a'), (1, 'b'). `range(5)` produit 0 à 4 à la demande, sans créer de liste.

Ce que fait `for` en coulisses (*rappel général*)

```
| data = ["a", "b"]
```

```

it = iter(data)
while True:
    try:
        item = next(it)
    except StopIteration:
        break
    print(item)

```

8. Structures de données et compréhensions

Source : C1, p. 11 à 14

Type	Nature, analogue C++	Particularités
list	tableau dynamique de pointeurs, mutable, ordonné; <code>std::vector<PyObject*></code>	indices négatifs; tranche <code>[start:end:step]</code> = nouvelle copie, fin exclue
tuple	liste immuable; <code>const</code> <code>std::vector</code> , <code>std::pair</code>	<code>(1,)</code> pour un élément; déballage <code>x, y = point</code> ; clé de dict possible
dict	table de hachage clé et valeur; <code>std::unordered_map</code>	clés hachables (immuables); <code>KeyError</code> si clé absente, <code>.get(k, défaut)</code> sinon
set	ensemble haché d'éléments uniques; <code>std::unordered_set</code>	<code>{}</code> crée un dict, écrire <code>set()</code> ; <code>remove</code> lève une erreur, <code>discard</code> non

Complexités données par le support

Opération de liste	Coût	Opération	Coût
<code>l[i]</code> , <code>len(l)</code>	$O(1)$	<code>l.insert(0, x)</code>	$O(N)$
<code>l.append(x)</code>	$O(1)$ amorti	<code>l.pop(0)</code>	$O(N)$
<code>l.pop()</code>	$O(1)$	<code>l.remove(v)</code>	$O(N)$
accès <code>d[k]</code>	$O(1)$ en moyenne		

Insérer ou retirer en tête décale tous les pointeurs, d'où $O(N)$. Le test `x in liste` parcourt la liste, $O(N)$; `x in set` passe par le hachage, $O(1)$ en

moyenne (*rappel général*). C'est l'objet du devoir 3 (p. 19) : mesurer `-1 in data` sur une liste et sur un ensemble de 0 à 10 000 000.

Mesure faite sur la machine de rédaction, 1 000 000 d'éléments

`-1 in liste` : environ 5 ms par test; `-1 in ensemble` : environ 0,1 μ s.
L'ordre de grandeur, pas la valeur, est à retenir : l'écart croît avec N .

Opérations d'ensemble et compréhensions

`a | b` ou `a.union(b)`, `a & b` ou `a.intersection(b)`. Les compréhensions sont en général plus rapides que la boucle équivalente, car la boucle est gérée en C (C1, p. 13 et 14).

```
nums = [1, 2, 3, 4, 5]
squares = [x**2 for x in nums if x % 2 == 0]          # [4, 16]
name_len = {n: len(n) for n in ["alice", "bob"]}
# name_len vaut {'alice': 5, 'bob': 3}
```

Le support laisse le résultat de `squares` en blanc (p. 14) ; la valeur ci-dessus a été obtenue à l'exécution.

9. Environnements virtuels et dépendances

Source : C1, p. 14 à 16 ; devoir 1, p. 19

L'interpréteur global est « sacré » : on isole chaque projet dans un environnement virtuel. Un **venv** est une arborescence autonome qui contient une copie (ou un lien) du binaire Python et son propre dossier `site-packages`. Le module `venv` fait partie de la bibliothèque standard depuis Python 3.3 ; sur Ubuntu et Debian il faut parfois installer `python3-venv`. Environnement actif : `python` lance le binaire local et `pip install` écrit dans le `site-packages` local.

```
python -m venv ai_env          # 1. créer
source ai_env/bin/activate     # 2. activer (Linux, Mac)
.\ai_env\Scripts\Activate.ps1 # activer (PowerShell)
which python                  # doit pointer dans ai_env
pip install --upgrade pip      # 3. installer
pip install numpy pandas matplotlib jupyterlab torch
pip freeze > requirements.txt  # 4. figer les versions
pip install -r requirements.txt # reproduire ailleurs
```

- **Activer** modifie la variable \$PATH pour que le shell cherche dans ai_env/bin avant /usr/bin.
- **pip** se connecte à PyPI, télécharge le paquet (souvent une roue .whl) et l'extrait dans ai_env/lib/python3.x/site-packages/.
- `import numpy` consulte `sys.path`, où le site-packages de l'environnement actif est en tête.
- `pip freeze` liste les paquets installés et leurs versions : reproductibilité, fin des bogues « chez moi ça marche ».

Le schéma p. 16 résume : système d'exploitation, Python global (« Do Not Touch »), dossier projet contenant ai_env avec bin/activate et site-packages/numpy, et les quatre étapes Create, Activate, Install, Freeze.

Piège

Le 01-requirements.txt distribué est encodé en UTF-16 avec fins de ligne Windows : le convertir en UTF-8 avant `pip install -r`. Il contient `pywinpty`, paquet propre à Windows (source : fiche CLAUDE.md et lecture du fichier).

10. La spirale en Python pur et ses limites

Source : C1, p. 16 à 18; C2, p. 32 et 33

On génère des points (x,y) de 3 classes disposées en bras de spirale : un jeu **non linéairement séparable**, souvent utilisé pour tester des réseaux de neurones. Pour la classe c et le point i parmi n :

$$\text{rate} = \frac{i}{n}, \quad r = 5 \cdot \text{rate}, \quad x = r \sin \theta, \quad y = r \cos \theta, \\ \theta = 4 \cdot \text{rate} + 4c + \mathcal{N}(0, \text{noise}).$$

```
for c in range(classes):
    for i in range(n_points):
        rate = i / n_points
        radius = rate * 5.0
        theta = (rate * 4.0) + (c * 4.0) + random.gauss(0, noise)
        X.append([radius * math.sin(theta),
                  radius * math.cos(theta)])
        y.append(c)
```

La visualisation optionnelle (p. 17 et 18) sépare les coordonnées par compréhension puis appelle `plt.scatter(x_val, y_val, c=data_y, cmap='brg', ...)`.

Pourquoi ce n'est pas assez (C1, p. 18 ; C2, p. 33)

1. **Mémoire fragmentée** : `X` est une liste de listes, soit des pointeurs vers des pointeurs vers des `PyFloatObject` dispersés sur le tas. Conséquence : défauts de cache constants (la « soupe »).
2. **Surcoût de l'interpréteur** : `math.sin` et `random.gauss` sont appelés scalaire par scalaire, avec vérification de type et recherche de méthode, $N \times K$ fois.
3. **GIL** : même en multithread, ces calculs ne s'exécutent pas en parallèle car ils restent dans la boucle de bytecode.
4. **Pas de GPU** : une liste Python ne peut pas être envoyée sur un GPU.

Annonce du cours 2 : remplacer la fonction par NumPy (mémoire contiguë, opérations vectorisées en C), avec un gain annoncé de 100 à 1000 fois.

11. Fonctions, classes et visibilité

Source : C2, p. 1 à 5 ; NB2

Fonctions. Objets de première classe : on peut les passer en argument, les renvoyer, les affecter à une variable. Arguments par défaut (`weight=1.0`), docstring, retour multiple par tuple (`return accuracy, loss` puis débalage).

Classes. Pas de déclaration des attributs au niveau de la classe : ils sont créés dans `__init__` en les attachant à `self`. `self` joue le rôle de `this`, mais doit figurer explicitement en premier paramètre de chaque méthode d'instance. Héritage : `class LinearLayer(BaseModule)`, appel du parent par `super().__init__()`.

Visibilité par convention (« we are all consenting adults »)

Nom	Effet
value	public, comportement par défaut ; il n'y a pas de mot-clé public
_value	« protégé » : simple indication d'usage interne, aucune contrainte
__value	« privé » par name mangling : renommé en <code>_NomClasse__value</code> . But : éviter les collisions de noms dans l'héritage, pas cacher

Classe BankAccount du support, sorties obtenues

```
acc = BankAccount("Alice", 1000)
acc.__balance                # AttributeError
acc._BankAccount__balance    # 1000 (porte dérobée)
acc.balance                   # 1000 (getter @property)
acc.balance = -5              # ValueError (setter)
list(vars(acc))
# ['owner', '_branch_code', '_BankAccount__balance']
```

Le getter se déclare avec `@property`, le setter avec `@balance.setter` : on ajoute une validation sans changer la syntaxe d'accès.

Piège

En C++ ou Java, l'accès à un membre privé est refusé à la compilation. En Python, rien n'est interdit : `__x` est seulement renommé. De même, `layer.__id` lève `AttributeError` alors que `layer._BaseModule__id` fonctionne (p. 5) : le préfixe est le nom de la classe où l'attribut est défini, pas celui de la sous-classe.

12. Cycle de vie d'un objet : `__new__` et `__init__`

Source : C2, p. 5 et 6

En C++, le constructeur alloue et initialise. Python sépare les deux rôles.

	<code>__new__</code>	<code>__init__</code>
Rôle	constructeur : crée l'instance	initialiseur : remplit l'instance
Premier argument	<code>cls</code> (la classe)	<code>self</code> (l'instance)
Valeur de retour	doit renvoyer une nouvelle instance	doit renvoyer <code>None</code>
Usage	rare : types immuables, singletons	courant
Ordre	appelé en premier (méthode statique)	appelé ensuite, si <code>__new__</code> a réussi

Deux cas où `__new__` est indispensable.

- Sous-classer un type immuable (`int`, `tuple`) : la valeur ne peut plus changer une fois dans `__init__`.
- Singleton : `__new__` vérifie si une instance existe et la renvoie au lieu d'en créer une ; impossible dans `__init__`, l'allocation a déjà eu lieu.

Singleton (*rappel général*), exécuté

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

print(Singleton() is Singleton()) # True
```

13. Héritage multiple, MRO et linéarisation C3

Source : C2, p. 6 à 11 ; NB2

Python accepte le vrai héritage multiple (Java ne l'autorise que par les interfaces). D'où le **problème du diamant** : si D hérite de B et C, qui héritent tous deux de A, quelle version de A est appelée et dans quel ordre ? Python tranche de façon déterministe avec l'algorithme de **linéarisation C3**, qui produit la **MRO** (Method Resolution Order), calculée à la création de la classe.

C3 garantit :

1. un ordre de recherche cohérent ;
2. la préséance locale : les parents sont parcourus dans l'ordre où ils sont écrits ;
3. la monotonie : si A précède B dans une linéarisation, cet ordre vaut dans toutes les sous-classes.

Les enfants passent toujours avant les parents. Si la hiérarchie est incohérente, Python lève `TypeError` à la définition de la classe.

`super()` est un **proxy dynamique** : il n'appelle pas « le parent » mais la classe suivante dans la MRO de l'objet. Dans un diamant, `super()` dans `Camera` peut appeler `Phone`. La décision ne dépend ni du type ni du nombre des arguments : il n'y a pas de surcharge par signature comme en C++ (encadré p. 8).

Diamant du support : `class SmartPhone(Camera, Phone)`

MRO (schéma p. 8, confirmée à l'exécution) : `SmartPhone`, `Camera`, `Phone`, `Device`, `object`. Version naïve où chaque `__init__` a une signature fixe, sortie obtenue :

```
Camera Init start for iPhone
TypeError: Phone.__init__() missing 1 required
        positional argument: 'number'
```

`SmartPhone` appelle `Camera` avec `(name, resolution)` ; dans `Camera`, `super().__init__(name)` appelle la classe suivante de la MRO, `Phone`, qui exige aussi `number`. C'est le piège des « signatures incompatibles ».

Piège

Avec `class SmartPhone(Phone, Camera)`, le support dit que Python lève immédiatement une erreur en appelant `Phone.__init__` avec `(name, resolution)`. À l'exécution, cet appel passe (la résolution est prise comme `number`, par position) ; l'erreur survient un cran plus loin, sortie obtenue :

```
TypeError: Camera.__init__() missing 1 required
positional argument: 'resolution'
```

La conclusion du support tient (ça casse), le lieu exact de l'erreur diffère.

Hiérarchie incohérente, sortie obtenue

```
class A: pass
class B(A): pass
class C(A, B): pass
# TypeError: Cannot create a consistent method
# resolution order (MRO) for bases A, B
```

`C(A, B)` demande A avant B, alors que B, sous-classe de A, doit précéder A.

14. Héritage coopératif : `*args` et `**kwargs`

Source : C2, p. 9 à 11 ; NB2

Pour qu'une chaîne `super()` fonctionne avec des signatures différentes, le motif standard est de passer des arguments nommés partout. Chaque classe prélève ce qui la concerne et transmet le reste : c'est l'**héritage coopératif**, que le support qualifie aussi d'injection de dépendances à l'exécution.

- `*args` : arguments positionnels en nombre variable, reçus comme un **tuple**.
- `**kwargs` : arguments nommés en nombre variable, reçus comme un **dictionnaire**. « `kwargs` » est une convention, pas un mot-clé.
- À l'appel, `f(**config)` déballe un dictionnaire : `f(**{'timeout': 30})` équivaut à `f(timeout=30)`.

```

class Device:
    def __init__(self, name, **kwargs):
        self.name = name
        super().__init__(**kwargs)
class Camera(Device):
    def __init__(self, resolution, **kwargs):
        self.resolution = resolution
        super().__init__(**kwargs)
class Phone(Device):
    def __init__(self, number, **kwargs):
        self.number = number
        super().__init__(**kwargs)
class SmartPhone(Camera, Phone):
    def __init__(self, resolution, number, name):
        super().__init__(resolution=resolution,
                        number=number, name=name)

```

Sortie obtenue avec les print du support :

```

>>> SmartPhone(resolution="12MP", number="555-0199",
...             name="Pixel 8")
Initializing SmartPhone...
Camera ready: 12MP
Phone radio ready: 555-0199
Device initialized: Pixel 8

```

L'ordre suit la MRO.

À retenir

Coopérativité : **chaque** classe de la chaîne doit appeler `super()`, même celle qui croit hériter directement d'`object`. Si `Camera` appelait `Device.__init__(self)` en dur, la chaîne serait rompue et `Phone.__init__` ne serait jamais appelé (C2, p. 11).

15. Méthodes magiques et fermetures

Source : C2, p. 11 et 12; NB2

`__call__` : l'objet callable

Les méthodes magiques (dunder) permettent aux objets d'utiliser les opérateurs du langage. `__call__` rend une instance callable comme une fonction, à la manière de `operator()` et des foncteurs C++. C'est vital en IA : un modèle est à la fois un objet avec un état (poids, gradients) et une fonction.

```
class Model:
    def __init__(self, weights):
        self.weights = weights
    def __call__(self, x):
        return x * self.weights

nn = Model(0.5)
print(nn(10))          # 5.0 (sortie obtenue)
```

C'est ce mécanisme qui fait qu'en PyTorch `model(x)` appelle `forward(x)` (C3, p. 7).

Fermetures (closures)

Une fermeture est une fonction qui garde l'accès aux variables de sa portée lexicale même appelée hors de cette portée. Analogie C++ : un lambda avec capture [&]. Différence : Python capture la *variable* (la liaison), pas la valeur ; la variable est déplacée de la pile vers le tas dans un **objet cellule** (cell object), dont la durée de vie est prolongée.

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter = make_counter()
print(counter(), counter())  # 1 2
import inspect
print(inspect.getclosurevars(counter))
```

```
# ClosureVars(nonlocals={'count': 2}, globals={},
#                 builtins={}, unbound=set())
```

Piège

Sans `nonlocal`, `count += 1` rend `count` locale à `increment`. Sortie obtenue :

```
UnboundLocalError: cannot access local variable
'count' where it is not associated with a value
```

Conséquence de la capture par liaison (« late binding ») : `[lambda: i for i in range(3)]` renvoie trois fonctions qui donnent toutes 2 (*rappel général*).

16. Décorateurs et métaprogrammation

Source : C2, p. 11 à 15 ; NB2

En C++, la métaprogrammation se fait à la compilation (templates). En Python, elle modifie fonctions et classes à l'exécution, surtout par les décorateurs, qui reposent sur les fermetures. Un décorateur est du sucre syntaxique pour une **fonction d'ordre supérieur** (qui prend une fonction et renvoie une fonction) :

```
@my_decorator          # est compilé exactement en :
def my_func():          #   def my_func(): pass
    pass                #   my_func = my_decorator(my_func)
```

Motif du support	Principe
Intercepteur (benchmark)	un wrapper(*args, **kwargs) mesure <code>time.perf_counter()</code> avant et après l'appel; profile une couche ou un chargeur de données sans polluer le code métier
Registre (fabrique)	<code>register_model(cls)</code> ajoute la classe dans <code>MODEL_REGISTRY[cls.__name__]</code> et la renvoie inchangée; <code>create_model("ResNet50")</code> instancie par nom. Motif de Hugging Face Transformers et MMDetection, qui remplace une cascade de if
Classe décoratrice (CountCalls)	la classe reçoit la fonction dans <code>__init__</code> et implémente <code>__call__</code> ; utile quand le décorateur garde un état (compteur, cache)

```
def benchmark(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"[{func.__name__}] executed in "
              f"{time.perf_counter() - start:.4f} seconds")
        return result
    return wrapper
```

À retenir

Toujours `@functools.wraps(func)` (ou `functools.update_wrapper` dans une classe décoratrice). Sans lui, la fonction décorée perd son `__name__` et son `__doc__` (elle devient wrapper) : débogueurs, IDE et sérialisation (pickle, utilisé en multiprocessing) cassent (C2, p. 15). Vérifié : sans wraps, `f.__name__` vaut wrapper et `f.__doc__` vaut None.

17. Google Colab et architecture Jupyter

Source : C2, p. 15 à 17

Colab est une interface de gestion de machines virtuelles Linux (Ubuntu) éphémères : tout ce qui est écrit sur le disque local hors d'un lecteur monté est perdu à la fin de la session.

Ressource	Gratuit (estimation)	Pro, Pro +
RAM	environ 12 Go	32 à 52 Go
GPU	NVIDIA T4 partagé	P100, T4, A100
Durée max	12 h	24 h
Inactivité	délai court	long, exécution en arrière-plan (Pro+)

Persistance. Colab monte Google Drive par **FUSE** (Filesystem in Userspace) : les opérations POSIX (open, read) marchent, mais chacune est un appel d'API réseau, donc une latence élevée face à un SSD local.

Architecture Jupyter

Trois notions à distinguer : l'**architecture** (conception client-serveur découplée), le **Notebook** (interface web, le frontend ; Colab en est un) et le **protocole de messagerie** (messages JSON entre frontend et noyau, indépendants du langage). Le **noyau** (kernel) est le backend qui exécute le code (Python, R, Julia). Transport : **ZeroMQ** (OMQ), bibliothèque de messagerie asynchrone.

Socket	Motif (schéma p. 17)	Rôle
Shell	ROUTER/DEALER	requêtes et réponses synchrones : exécution de code, introspection
IOPub	PUB/SUB	diffusion asynchrone des effets : stdout, erreurs, graphiques
Stdin	ROUTER/DEALER	entrée bloquante de l'utilisateur (input())
Control	ROUTER/DEALER	canal prioritaire : interruption, arrêt, même si Shell est occupé
Heartbeat	REQ/REP	écho qui vérifie que le noyau répond

Le schéma p. 17 place un serveur Jupyter (gestionnaire de processus, proxy) entre le frontend (navigateur, relié en HTTP et WebSockets) et le noyau (par exemple IPyKernel).

Piège

Le texte annonce « cinq sockets » : Shell, IOPub, Stdin, Control, Heartbeat. Interrompre un calcul long passe par Control, pas par Shell (occupé). Un print arrive par IOPub.

18. ndarray : buffer, métadonnées et strides

Source : C2, p. 17 à 19

NumPy est un moteur de gestion mémoire qui expose au Python les unités vectorielles du CPU. Le nom **ndarray** souligne deux différences avec les conteneurs usuels :

1. **homogénéité** : chaque élément occupe exactement le même nombre d'octets (contrairement à une liste Python ou un `std::tuple`) ;
2. **rang arbitraire** : un tableau 2D n'est pas un tableau de tableaux mais un **bloc contigu unique** (tampon 1D) ; les dimensions sont une abstraction logique définie par les métadonnées.

Un ndarray = un **tampon de données brut** (comme un `float* C`) + des **métadonnées** : dtype, shape et surtout strides (octets à sauter pour avan-

cer d'un cran sur chaque axe).

À retenir

Formule d'adressage (p. 18) : l'élément d'indice (i, j, k) , avec les strides (s_0, s_1, s_2) , est à l'octet

$$\text{Offset} = i \cdot s_0 + j \cdot s_1 + k \cdot s_2.$$

Pour un tableau contigu en ordre C (*rappel général*) : le dernier stride vaut la taille d'un élément, et chaque stride est le produit du suivant par la dimension suivante.

Exemple du support, sorties obtenues

```
arr = np.array([[5, 6, 7], [8, 9, 10]], dtype=np.int32)
arr.shape, arr.strides      # (2, 3) (12, 4)
arr.T.shape, arr.T.strides  # (3, 2) (4, 12)
np.shares_memory(arr, arr.T) # True
```

int32 = 4 octets : 4 pour passer à la colonne suivante, $3 \times 4 = 12$ pour la ligne suivante. La transposée échange shape et strides sans toucher au tampon [5, 6, 7, 8, 9, 10] (schéma p. 19).

Calcul d'offset

`np.zeros((2, 3, 4), dtype=np.float32)` : strides $(3 \cdot 4 \cdot 4, 4 \cdot 4, 4) = (48, 16, 4)$. L'élément (1, 2, 3) est à $48 + 2 \cdot 16 + 3 \cdot 4 = 92$ octets. Même tableau en float64 : (96, 32, 8).

19. Vues et copies : slicing et fancy indexing

Source : C2, p. 18 à 20

Comme les métadonnées sont séparées du tampon, beaucoup d'opérations sont en $O(1)$ sans déplacement de données : **transposée** (`.T`, échange shape et strides) et **reshape** (nouvelle shape si les données s'y prêtent sans copie). La même syntaxe `[]` recouvre deux opérations radicalement différentes selon le type de l'indice.

	Slicing de base	Indexation avancée (fancy)
Indice	entiers, tranches <code>start:stop:step, ...</code>	liste ou tableau d'entiers, masque booléen
Résultat	vue : nouvel en-tête sur le même tampon	copie : nouveau bloc alloué
Coût	$O(1)$ quelle que soit la taille	$O(N)$ en nombre d'éléments
Modifier le résultat	modifie l'original	ne touche pas l'original

Pourquoi une copie : les positions demandées ne sont ni contiguës ni périodiques, aucune arithmétique de strides ne les décrit ; NumPy « rassemble » (gather) les valeurs dans un bloc contigu. Un **masque booléen** `arr > 0.5` est un tableau de même forme rempli de `True/False` ; `arr[mask]` renvoie toujours un tableau **1D**, car les `True` peuvent être dispersés et la forme rectangulaire n'est pas garantie.

Sorties obtenues

```
big = np.zeros((1000, 1000))
sub = big[0:500, :]
np.shares_memory(big, sub)      # True
sub[0, 0] = 7; big[0, 0]         # 7.0 : la vue écrit dans big
sel = big[[1, 5, 2]]
np.shares_memory(big, sel)      # False ; sel.shape (3, 1000)
m = np.arange(12).reshape(3, 4)
m[m > 5]                         # [ 6  7  8  9 10 11], forme (6,)
```

Piège

Deux pièges opposés. Modifier une tranche « pour tester » modifie les données d'origine. À l'inverse, `arr[mask][0] = 0` ne modifie rien : on écrit dans une copie (`arr[mask] = 0` en affectation directe, lui, écrit dans `arr` (*rappel général*)).

20. SIMD et vectorisation

Source : C2, p. 21 ; p. 35 et 36

Via ses bibliothèques C et Fortran (BLAS, MKL), NumPy descend jusqu'aux instructions **SIMD** (Single Instruction, Multiple Data). Boucle classique (SISD) : charger un nombre, l'additionner, le stocker. SIMD : charger un vecteur de nombres dans un registre large et tous les additionner en un cycle.

Jeu	Registre	Flottants 32 bits	Origine
SSE	128 bits	4	Intel, 1999
AVX	256 bits	8	Intel et AMD, 2011
AVX-512	512 bits	16 par cycle	serveurs et CPU Intel haut de gamme
NEON	128 bits	4	ARM (Apple M1 à M3, mobiles)

Règle de calcul : nombre de flottants = largeur du registre / 32.

Déroulé de `C = A + B` (p. 21) : Python ne boucle pas ; il délègue à une fonction C ; celle-ci détecte les capacités du CPU à l'exécution ; avec AVX2, elle charge 8 flottants de A et 8 de B dans des registres YMM, exécute `vaddps` (Vector Add Packed Single) et écrit le résultat. Gain théorique annoncé : 8 à 16 fois sur du C++ scalaire, 1000 fois sur une boucle Python.

Piège

Le support (p. 36) dit que `np.sin` applique une instruction CPU « `vsinps` ». Les jeux x86 cités n'ont pas, à notre connaissance, d'instruction matérielle de sinus vectoriel : le sinus est calculé par une routine

vectorisée. Ce point est **À VÉRIFIER** ; retenir l'idée du support : une boucle C sur le tampon, plusieurs valeurs par instruction.

21. Broadcasting

Source : C2, p. 21 à 23

Le broadcasting est l'algorithme déterministe qui permet l'arithmétique entre tableaux de formes différentes, en « étirant » virtuellement des dimensions sans copier de données.

À retenir

1. **Aligner** les formes à droite (dernière dimension d'abord).
2. **Compléter** la forme la plus courte par des 1 à gauche.
3. **Comparer** dimension par dimension : compatibles si égales ou si l'une vaut 1.
4. **Étirer** chaque 1 à la taille de l'autre ; résultat = maximum de chaque paire.

L'étirement est implémenté par un **stride nul** : le CPU relit la même adresse.

Les deux cas du support, vérifiés

Matrice (4, 3) + vecteur (3,) : le vecteur devient (1, 3) ; $3 = 3$, et 1 s'étire en 4 ; résultat (4, 3), le vecteur est ajouté à chaque ligne.

(4, 3) + (4,) : complété en (1, 4) ; dernière dimension $3 \neq 4$: erreur.

```
ValueError: operands could not be broadcast  
together with shapes (4,3) (4,)
```

Correction : `B.reshape(4, 1)`, dont le 1 s'étire en 3 ; résultat (4, 3).

Stride nul, sortie obtenue

`np.broadcast_to(np.array([1., 2., 3.]), (4, 3)).strides`
vaut (0, 8) : avancer d'une ligne ne déplace pas le pointeur.

A	B	Résultat
(100, 10)	(1, 10)	(100, 10) (biais du cours)
(100, 50)	(1, 50)	(100, 50) (normalisation)
(8, 1, 6, 1)	(7, 1, 5)	(8, 7, 6, 5)
(2, 3)	(3, 2)	erreur : $3 \neq 2$

Les deux dernières lignes sont des exemples (*rappel général*) vérifiés avec NumPy 2.5.

22. Fonctions NumPy pour un pipeline de ML

Source : C2, p. 23 à 27

Création et génération

- `np.random.randn(100, 50)` : 100 échantillons, 50 caractéristiques, loi normale centrée réduite. Poids `np.random.randn(50, 10) * 0.01`, biais `np.zeros((1, 10))` : la forme (1,10) permet le broadcasting.
- `np.linspace(start, stop, num=50)` divise l'intervalle en $N - 1$ parts égales, sans addition itérative :

$$x_i = \text{start} + i \cdot \frac{\text{stop} - \text{start}}{\text{num} - 1}.$$

Borne finale garantie (sauf `endpoint=False`), écriture directe dans un bloc pré-alloué. `np.arange(start, stop, step)`, lui, additionne un pas ; si le pas n'est pas représentable exactement en binaire (0,1), l'erreur s'accumule et le dernier élément peut « déborder ».

- `np.random.randn` : loi normale standard, $\mu = 0$, $\sigma = 1$,

$$p(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

`np.random.rand` donne une loi uniforme sur $[0, 1)$. Sous le capot : allocation contiguë, générateur pseudo-aléatoire (par exemple PCG64) avec Box-Muller ou Ziggurat, sans objet Python par nombre, bien plus rapide que `random.gauss()` en boucle.

- **Brisure de symétrie** : des poids tous nuls donneraient des gradients symétriques, les neurones « bougeraient ensemble » ; l'aléa de `randn` l'évite. On ajuste l'échelle (par exemple $\times 0,01$) selon la profondeur (le support cite l'initialisation de He).

Sorties obtenues

```
np.linspace(0, 1.0, 11) # [0. 0.1 0.2 ... 0.9 1. ]
np.linspace(0, 10, 5)   # [ 0.  2.5  5.  7.5 10. ]
np.arange(1, 1.3, 0.1)  # [1.  1.1 1.2 1.3]
```

Le dernier cas illustre la dérive annoncée par le support : la borne stop est censée être exclue, l'accumulation d'erreurs la fait apparaître.

Prétraitement, passe avant, prédiction

```
mean = np.mean(X, axis=0, keepdims=True) # (1, 50)
std  = np.std(X, axis=0, keepdims=True)  # (1, 50)
X_norm = (X - mean) / (std + 1e-8)        # z-score
Z = np.dot(X_norm, W)                    # (100,50)@(50,10)
Z = Z + b                                # biais diffusé
A = np.maximum(0, Z)                     # ReLU
predictions = np.argmax(A, axis=1)       # (100,)
flat = images.reshape(32, -1)            # (32,28,28,3) -> (32, 2352)
```

- `axis=0` « écrase » la dimension 0 (les lignes) : une statistique par colonne (caractéristique). `axis=1` : une valeur par ligne (échantillon).
- `keepdims=True` garde la dimension écrasée à taille 1 : (1, 50) au lieu de (50,), prêt pour le broadcasting.
- 10^{-8} évite la division par zéro pour une colonne constante.
- Dans `np.maximum(0, Z)`, le 0 est diffusé à la forme de Z ; ReLU annule les valeurs négatives.
- -1 dans `reshape` : « déduis cette dimension de la taille totale », ici $28 \times 28 \times 3 = 2352$.

23. Pandas : moteur en colonnes pour les données

Source : C2, p. 27 à 30 ; NB1

Les données réelles sont rarement propres ou homogènes (valeurs manquantes, chaînes catégorielles, types variés). NumPy calcule sur des matrices homogènes ; Pandas est l'infrastructure en colonnes des pipelines ETL (Extract, Transform, Load) et de feature engineering, le pont entre l'ingestion brute et les tenseurs.

Pandas 3.0 et Apache Arrow (selon le support) : chaque colonne dans un bloc mémoire continu isolé, ajout de colonnes sans recopier tout le jeu ; texte et catégories traités nativement en tampons UTF-8 continus, extraction de caractéristiques textuelles « jusqu'à six fois » plus rapide ; **Copy-on-Write** par défaut : une sélection ne duplique que des métadonnées, la copie physique n'a lieu qu'à la première modification.

Attribut	Rôle
<code>.index</code>	étiquettes des lignes, identifiants uniques
<code>.columns</code>	étiquettes des colonnes
<code>.values</code>	données brutes en tableau NumPy (perd la protection de l'index)
<code>.dtypes</code> , <code>.shape</code>	type de chaque colonne ; (lignes, colonnes)
<code>.loc</code> , <code>.iloc</code>	sélection par étiquette ; par position entière

Un DataFrame est fait de colonnes, chacune une **Series** ; il peut être hétérogène (int, float, chaîne dans le même objet).

```
raw["income"] = raw["income"].fillna(value=raw["income"].median())
raw["status"] = raw["status"].astype(dtype="category")
features = raw.drop(columns=["user_id"])
```

- **Imputation** : `fillna` remplace les NaN (la plupart des modèles refusent les valeurs manquantes) ; on garde l'échantillon au lieu de le supprimer. Variantes : `ffill` (dernière valeur valide vers l'avant), `bfill` (valeur suivante vers l'arrière), `interpolate`, `dropna` (supprime lignes ou colonnes), `replace`.

- **category** : dictionnaire interne des valeurs uniques et stockage d'entiers ; pour une faible cardinalité, mémoire réduite de 90 à 95 %, regroupements et tris plus rapides.
- **drop(columns=["user_id"])** : un identifiant n'est pas prédictif ; le garder provoque fuite de données et mémorisation.

Piège

Deux affirmations du support sont **À VÉRIFIER** avant de les réciter mot pour mot : que Pandas 3.0 stocke toutes les colonnes en Arrow (la documentation Pandas parle surtout d'un type chaîne adossé à PyArrow par défaut), et l'existence de `torch.from_arrow` (concept 24).

24. Alignement par index et passage aux tenseurs

Source : C2, p. 30 à 32

Fusionner caractéristiques et étiquettes *par position* risque une corruption silencieuse si une observation disparaît du pipeline. Pandas aligne sur les **étiquettes d'index** dans toutes les opérations et jointures : une clé présente d'un côté et absente de l'autre reçoit une valeur manquante, au lieu de décaler les lignes suivantes.

```
features = pd.DataFrame({"clicks": [1, 2, 3]},
                        index=["u1", "u2", "u3"])
labels = pd.Series([1.0, 0.0], index=["u3", "u1"],
                  name="converted")
training = features.join(labels, how="left")
```

D'après le support, u2 reçoit NaN au lieu d'hériter de l'étiquette de u3 ; u1 reçoit 0.0 et u3 1.0, malgré l'ordre différent. Sortie non reproduite ici (Pandas absent de la machine de rédaction).

Interopérabilité sans copie

Le support décrit le protocole **Arrow PyCapsule** : Pandas exposerait ses adresses mémoire au niveau C à PyTorch ou TensorFlow, pour créer un tenseur sans octet supplémentaire, avec la fonction `torch.from_arrow(df)`. Nom de fonction **À VÉRIFIER** (voir concept 23). Le pont éprouvé dans les

trois cours est `torch.from_numpy()`, qui partage le tampon d'un tableau NumPy (C2, p. 35 et 37 ; C3, p. 16).

Pourquoi l'alignement compte

Si on concaténait par position les clics `[1, 2, 3]` et les étiquettes `[1.0, 0.0]`, `u1` recevrait 1.0 (l'étiquette de `u3`) et `u2` 0.0 : des données d'entraînement fausses sans aucune erreur. La jointure par index l'empêche.

25. Spirale vectorisée, GIL et choix des outils

Source : C2, p. 32 à 37 ; NB2

Modèle vectorisé (p. 33 et 34). Pour le vecteur d'indices $I = [0, 1, \dots, n]$, on calcule tout un bras de spirale d'un coup :

$$R = \frac{I}{n} \cdot 5, \quad \Theta_c = \frac{I}{n} \cdot 4 + 4c + \mathcal{N}(0, \text{noise}).$$

```
class SpiralDataset:
    def __init__(self, n_points_per_class, n_classes=3):
        self.n_points = n_points_per_class
        self.n_classes = n_classes
        total = n_points_per_class * n_classes
        self.X = np.zeros((total, 2), dtype=np.float32)
        self.y = np.zeros(total, dtype=np.uint8)
    def generate(self, noise=0.2):
        for c in range(self.n_classes):
            ix = range(self.n_points * c,
                        self.n_points * (c + 1))
            r = np.linspace(0.0, 5.0, self.n_points)
            t = (np.linspace(c * 4, (c + 1) * 4, self.n_points)
                 + np.random.randn(self.n_points) * noise)
            self.X[ix] = np.c_[r * np.sin(t), r * np.cos(t)]
            self.y[ix] = c
        return self.X, self.y
```

Avec 1000 points par classe, sortie obtenue : `X.shape = (3000, 2)`, `X.strides = (8, 4)` (4 octets par float32, 8 pour une ligne de deux). `np.c_` empile deux vecteurs 1D en colonnes d'une matrice ; l'affectation écrit dans les lignes `ix` du tampon pré-alloué. Il reste une boucle Python sur les 3 classes, aucune sur les points.

Critère (p. 35, 36)	Python pur	NumPy
Exécution	interpréteur CPython (à pile)	C et Fortran compilés (registres)
Mémoire	« soupe » de PyObject fragmentée	tampon contigu avec strides
Concurrence	bloquée par le GIL	GIL relâché ; BLAS, LAPACK multithread
Calcul	$O(N)$ dispatchs Python	SIMD (AVX-512, NEON)
Accès	indirection de pointeurs (lent)	arithmétique de pointeurs (rapide)

GIL (Global Interpreter Lock, p. 37) : empêche plusieurs threads d’exécuter du bytecode en même temps, pour protéger le comptage de références. Les extensions C de NumPy et Pandas le **relâchent** pendant les calculs lourds, ce qui laisse BLAS et LAPACK utiliser plusieurs cœurs : le code vectorisé est la seule voie vers le multicœur en CPython sans le module multiprocessing.

Choix des outils (p. 37) : ingestion (CSV, Parquet, SQL, schéma hétérogène) et nettoyage et jointures (alignement par index) : Pandas ; algèbre linéaire (SIMD, BLAS, LAPACK) et entrées du modèle (strides, reshape et batch sans copie) : NumPy. Conclusion du cours : garder les données contiguës et homogènes le plus longtemps possible, Python ne servant qu’à « dispatcher » des opérations en bloc.

26. Matériel : CPU, GPU, SIMT et tenseur

Source : C3, p. 1 et 2

Le deep learning remplace une logique séquentielle à branchements par des transformations massivement parallèles d’espaces géométriques. Les hiérarchies d’objets profondes sont scalaires et provoquent des défauts de cache : il faut aligner les structures de données sur les contraintes du matériel.

	CPU	GPU
Optimisé pour	la latence	le débit
Silicium consacré à	prédiction de branchement, exécution dans le désordre	très nombreuses petites ALU
Fait pour	système d'exploitation, flot de contrôle complexe	la même opération sur des millions de données
Schéma p. 1	unité de contrôle, grosses ALU, caches L1, L2, L3, DRAM	multiprocesseurs (SM), mémoire partagée et cache L1, registres, VRAM globale

À retenir

- **SIMT** (Single Instruction, Multiple Threads) : une instruction diffusée à de nombreux threads qui l'exécutent en même temps sur des données différentes ; on privilégie le débit sur la latence d'un thread.
- **Warp** : groupe de threads (typiquement 32) qui exécutent la même instruction sur des adresses différentes.
- **Hiérarchie mémoire** : les données passent de la VRAM globale, lente, vers la SRAM (mémoire partagée, cache L1, accès en un cycle) et les registres, pour ne jamais affamer les ALU.
- **Tenseur** : bloc contigu de mémoire brute + métadonnées (dtype, shape, strides), qui permet des changements de topologie en $O(1)$ sans copie. C'est la notion de ndarray (concept 18) transposée aux frameworks.

27. PyTorch : philosophie et allocateur à cache

Source : C3, p. 2 et 3

PyTorch, développé principalement par Meta (Facebook AI Research), privilégie l'expérience du développeur et un style impératif « pythonique » : **le code Python est le modèle**, sans étape de compilation séparée pour construire le réseau.

Le goulet `cudaMalloc`

En C++, `std::malloc` délègue à l'allocateur du système. Sur GPU, `cudaMalloc` est « catastrophiquement lent » : passage de l'espace utilisateur à l'espace noyau et synchronisation globale CPU et GPU. Appelé pour chaque variable temporaire d'une boucle d'entraînement, il viderait le pipeline GPU à chaque fois.

La solution : le CUDA Caching Allocator

- Au démarrage, PyTorch demande au pilote de grands blocs contigus, qu'il gère lui-même en espace utilisateur (schéma p. 3 : « Internal Memory Pool »).
- Quand un tenseur est détruit par le ramasse-miettes, la mémoire n'est **pas rendue à CUDA** : elle reste dans le pool pour être recyclée immédiatement.
- Sémantique **ordonnée par stream** : un pointeur libéré peut être donné au noyau (kernel) suivant du même stream CUDA sans attendre la fin physique de l'exécution, donc sans bloquer le CPU.

Piège

Conséquence pratique (*rappel général*) : la mémoire GPU vue par `nvidia-smi` ne baisse pas quand on supprime un tenseur, car PyTorch la garde en cache. Côté TensorFlow, le comportement est l'inverse : allocation « gourmande » de toute la VRAM au démarrage (C3, p. 13, concept 33).

28. Autograd : différentiation automatique

Source : C3, p. 3 et 4

Un réseau apprend en optimisant des millions de paramètres par descente de gradient. Dériver à la main la perte par rapport à chaque paramètre est impossible ; il faut un système qui enregistre les opérations et les dérive automatiquement.

À retenir

- **Différentiation automatique en mode inverse** : une passe avant enregistre les opérations, puis on parcourt l'historique à l'envers, de la sortie vers l'entrée, en appliquant la règle de dérivation en chaîne.
- **Produit Jacobien-vecteur (JVP)** : au lieu de calculer la jacobienne complète, chaque nœud multiplie le gradient venu d'en amont par sa dérivée locale.
- Un réseau est un « entonnoir » : des millions d'entrées pour **une** perte scalaire. Le mode direct demanderait une passe par paramètre ; le mode inverse n'en demande qu'une seule vers l'arrière.

Le graphe dynamique (DAG, schéma p. 4). Chaque opération sur un tenseur marqué `requires_grad=True` alloue un nœud `Function` à la volée. Les poids apprenables sont les **feuilles** ; les opérations sont les nœuds internes, chacun avec sa fonction de dérivée. Le schéma : feuilles W , b , x ; nœud de multiplication, nœud d'addition, perte L au sommet ; flèches avant (données) et arrière (gradients $\partial L/\partial W$, $\partial L/\partial b$, $\partial L/\partial x$).

.`backward()` déclenche, dans le backend C++ (bibliothèque `ATen`), un tri topologique du graphe, applique la règle de chaîne, **accumule** les gradients dans l'attribut `.grad` des feuilles, puis détruit le graphe pour libérer la mémoire.

Règle de chaîne sur le schéma (rappel général)

$L = Wx + b$ (scalaires). $\partial L/\partial b = 1$, $\partial L/\partial W = x$, $\partial L/\partial x = W$. Avec $W = 3$, $x = 2$, $b = 1$: $L = 7$ et les gradients valent 1, 2 et 3. Si L alimente ensuite une perte $\ell = L^2$, le gradient amont $\partial \ell/\partial L = 2L = 14$ multiplie chaque dérivée locale (JVP) : $\partial \ell/\partial W = 14 \times 2 = 28$.

Piège

Le support écrit que le mode inverse atteint un temps « $O(1)$ par rapport au nombre de paramètres ». Lecture juste (*rappel général*) : le *nombre de passes* arrière ne dépend pas du nombre de paramètres (une seule), mais cette passe coûte de l'ordre d'une passe avant, qui elle grandit avec la taille du réseau. Et comme `.grad accumule`, il faut remettre les gradients à zéro à chaque itération (concept 30).

29. nn.Module, couches et activations

Source : C3, p. 4 à 7

`torch.nn.Module` est la classe de base de tout composant de réseau. Elle implémente le patron **Composite** : une couche et un réseau entier se manipulent de la même façon, récursivement. Motivation : détecter automatiquement les paramètres, les envoyer sur le GPU et les passer à l'optimiseur sans écrire d'accesseurs, par réflexion et métaprogrammation.

Mécanisme (schéma p. 5). `nn.Module` redéfinit `__setattr__`. Affecter un `nn.Parameter` (tenseur entraînable) ou un sous-module à `self` dans `__init__` le range dans des dictionnaires ordonnés spécialisés : `_parameters`, `_modules`, `_buffers` (état non entraînable, par exemple les statistiques de batch normalization).

Piège

Contourner cette interception (couches rangées dans une liste Python ordinaire, attribut posé sans enregistrement) cache les paramètres au framework : l'optimiseur ne les voit jamais et le modèle n'apprend pas (p. 5). De même, oublier `super().__init__()` empêche la création de ces dictionnaires (commentaire « CRITICAL » du code p. 6).

Brique	Rôle selon le support
<code>nn.Linear(in, out)</code>	couche dense : matrice de poids et biais, calcule $y = xA^T + b$
<code>nn.Conv2d()</code>	convolution spatiale pour l'image, filtres glissants
<code>nn.ReLU()</code>	$\max(0, x)$; défaut efficace des couches cachées, limite la disparition du gradient
<code>nn.LeakyReLU(slope)</code>	petite pente non nulle pour $x < 0$, évite les « neurones morts »
<code>nn.GELU()</code>	variante lisse et probabiliste de ReLU, utilisée dans GPT et BERT
<code>nn.Sigmoid()</code>	écrase dans $]0, 1[$; dernière couche en classification binaire
<code>nn.Softmax(dim)</code>	transforme des scores bruts (logits) en distribution de somme 1 ; multi-classe

Les activations apportent la non-linéarité sans laquelle un empilement de couches linéaires resterait linéaire et ne tracerait pas de frontières de décision courbes.

À retenir

Formules (*rappel général*) : $\sigma(z) = \frac{1}{1 + e^{-z}}$, $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$,
paramètres de `nn.Linear(in, out)` = $\text{in} \times \text{out} + \text{out}$.

Compter les paramètres du SimpleMLP du cours (*rappel général*)

SimpleMLP(10, 32, 2) : Linear(10, 32) a $320 + 32 = 352$ paramètres, GELU aucun, Linear(32, 2) en a $64 + 2 = 66$: total 418.

30. Boucle d'entraînement PyTorch

Source : C3, p. 6 et 7; p. 15 à 17; NB1, NB3

```
class SimpleMLP(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.activation = nn.GELU()
        self.layer2 = nn.Linear(hidden_size, num_classes)
    def forward(self, x):
        x = self.layer1(x)
        x = self.activation(x)
        return self.layer2(x)          # logits

model = SimpleMLP(10, 32, 2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
loss_function = nn.CrossEntropyLoss()
raw_output = model(torch.randn(16, 10)) # appelle forward
loss = loss_function(raw_output, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Étape	Code	Effet
A	<code>logits = model(X)</code>	passe avant, construit le DAG
B	<code>loss = criterion(...)</code>	perte : racine du DAG
C	<code>optimizer.zero_grad()</code>	efface les <code>.grad</code> des feuilles
D	<code>loss.backward()</code>	parcours arrière, calcul des gradients (JVP)
E	<code>optimizer.step()</code>	met à jour les poids d'après <code>.grad</code>

- Les hyperparamètres sont passés à des objets autonomes. L'optimiseur Adam reçoit `model.parameters()`, qui parcourt récursivement tous les paramètres enregistrés, et le taux d'apprentissage `lr`.
- `nn.CrossEntropyLoss` attend des **logits bruts**, pas des probabilités (commentaire p. 17) : pas de softmax en sortie du modèle.
- `predictions = raw_output.argmax(dim=1)` : la classe prédite par ligne.

- Mode entraînement ou évaluation : `model.eval()` coupe les comportements propres à l'entraînement (dropout), `model.train()` les rétablit.
- Sous `with torch.no_grad()` : aucun DAG n'est construit : économie de mémoire à l'évaluation.
- Précision : `(predicted == y).float().mean()`.

À retenir

Mise à jour de la descente de gradient (*rappel général*) : $\theta \leftarrow \theta - \eta \nabla_{\theta} L$, avec η le taux d'apprentissage (lr). Entropie croisée pour la bonne classe y (*rappel général*) : $L = -\log \text{softmax}(z)_y$.

Piège

- Oublier `zero_grad()` : les gradients s'additionnent d'une itération à l'autre et l'entraînement est détruit (p. 7 et 17). Il doit précéder `backward()`.
- Le code du cours appelle `.cuda()`, qui échoue sans GPU NVIDIA (cas d'un Mac). Écrire plutôt, comme en p. 16 :

```
device = torch.device("cuda" if torch.cuda.is_available()
                      else "cpu")
```

Dans NB1, le modèle reste sur CPU mais les étiquettes sont envoyées par `.cuda()` : ce mélange provoque une erreur de périphérique (*rappel général*).

- Ajouter `nn.Softmax` avant `CrossEntropyLoss` applique deux fois la normalisation (*rappel général*).

31. Mode eager, define-by-run et torch.compile

Source : C3, p. 7 et 8 ; p. 16 et 17

Define-by-Run (graphe dynamique) : le graphe se construit pendant l'exécution du code Python ; on peut utiliser le débogueur Python et un if/else ordinaire dans le réseau. C'est le **mode eager** historique de PyTorch : chaque opération lance son propre noyau GPU.

Problème : excellent pour déboguer, lent en production. Lancer un noyau CUDA pour chaque addition affame le GPU, car l'interpréteur Python n'émet pas les commandes assez vite. Il faut fusionner les opérations.

PyTorch 1	model(x) exécuté ligne à ligne ; la classe et son état dictent l'exécution
PyTorch 2	compiled_model = torch.compile(model) : compilateur JIT via TorchDynamo ; à l'appel, PyTorch intercepte l'exécution Python, extrait un arbre des opérations, les fusionne en noyaux C++ ou Triton optimisés et les exécute sur le GPU sans le surcoût Python

Le tableau comparatif (C3, p. 13) nomme le couple TorchDynamo et TorchInductor pour générer les noyaux Triton et C++.

Piège

Dans l'exemple de la spirale, `model = torch.compile(model)` est actif dans le PDF (p. 17) mais commenté dans NB3. `torch.compile` est une optimisation optionnelle : le modèle s'entraîne aussi sans (commentaire « Optional Optimization », p. 16).

32. TensorFlow et Keras

Source : C3, p. 8 à 11 ; NB3

TensorFlow (Google Brain) vise le déploiement distribué à grande échelle plus que l'expérimentation académique rapide.

Define-Then-Run (graphe statique) : la définition du modèle est séparée de son exécution. Le Python ne sert qu'à assembler un plan (le graphe), compilé puis exécuté indépendamment. Motivation : connaissant toute la topologie avant le passage des données, le compilateur applique des optimisations globales (constant folding, fusion d'opérateurs) impossibles en eager. Le graphe final est sérialisé en **Protocol Buffer** (GraphDef), sans dépendance à Python : exécution native en C++, en Java, sur mobile, dans le navigateur (schéma p. 9).

TensorFlow 2.x : le débogage d'un graphe statique était pénible (un print affichait les métadonnées du nœud, pas la valeur) ; TF2 passe en **eager par défaut** et intègre **Keras**, API de haut niveau créée par François Chollet, désormais interface officielle de TensorFlow.

Façon de définir	Usage
keras.Sequential API fonctionnelle	liste linéaire de couches, flux direct de A vers B topologies complexes (connexions résiduelles, branches, entrées ou sorties multiples) : Input, couches appelées comme des fonctions, puis keras.Model(inputs, outputs) ; construit un DAG de couches
Sous-classement	hériter de keras.Model et écrire call(), comme nn.Module

Couches : Dense(units, activation=...) (équivalent de nn.Linear), Conv2D, MaxPooling2D (vision), Dropout(rate) (met à zéro une fraction des entrées à l'entraînement contre le surapprentissage). Activations passées en chaîne (activation='relu') ou en objets.

Hyperparamètres intégrés au modèle via compile() et fit() : optimiseur (adam, rmsprop, sgd), taux d'apprentissage (le plus critique), taille de batch (échantillons traités avant chaque mise à jour), époques (passages sur tout le jeu).

```

inputs = keras.Input(shape=(10,))
x = layers.Dense(64, activation="relu")(inputs)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(2, activation="softmax")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=10, batch_size=32)
final = tf.argmax(model.predict(X_train[:5]), axis=1)

```

`fit()` cache passe avant, passe arrière et mise à jour. Ici la sortie est un softmax et la perte `sparse_categorical_crossentropy` prend des étiquettes entières.

Calcul (rappel général)

1000 échantillons, `batch_size=32` : $\lceil 1000/32 \rceil = 32$ mises à jour par époque (31 batchs pleins et un de 8), donc 320 sur 10 époques. Paramètres : Dense(64) sur 10 entrées = $640 + 64 = 704$; Dense(2) = $128 + 2 = 130$; Dropout aucun; total 834.

33. Comparer PyTorch et TensorFlow

Source : C3, p. 12 et 13

Critère	PyTorch	TensorFlow (Keras)
1. Exécution	dynamique (define-by-run) par défaut	hybride : eager par défaut, graphe statique via <code>@tf.function</code>
2. Ergonomie	très pythonique, contrôle de flux et débogueur natifs	déclaratif et structuré, adapté aux pipelines standards
3. API	<code>nn.Module</code> , <code>forward()</code> explicite	<code>keras.Sequential</code> ou <code>keras.Model</code>
4. Entraînement	boucle explicite (<code>zero_grad</code> , <code>backward</code> , <code>step</code>), contrôle total	<code>model.fit()</code> ; logique sur mesure via <code>tf.GradientTape</code>
5. Sérialisation	<code>state_dict</code> en <code>.pt</code> (pickle) : il faut Python et la classe d'origine	<code>SavedModel</code> (Protocol Buffers), graphe autonome
6. Compilation	<code>torch.compile</code> (TorchDynamo, TorchInductor, Triton, C++)	compilateur XLA
7. Déploiement	TorchScript, TorchServe ; historiquement plus dur sur l'embarqué	TF Serving, TF Lite (mobile), TF.js (navigateur)
8. Mémoire GPU	sobre : allocateur à cache par stream	gourmande : prend toute la VRAM au démarrage

À retenir

Fil directeur : PyTorch privilégie la flexibilité de recherche (le code est le modèle), TensorFlow l'industrialisation (le graphe est l'artefact déployé). Le résumé du cours (p. 18) présente ce compromis entre flexibilité de recherche et déploiement en production.

Question type

« Pourquoi charger un modèle PyTorch `.pt` demande-t-il le code source du modèle ? » Parce que le fichier contient un dictionnaire d'état séria-

lisé par pickle, pas le graphe : il faut la classe `nn.Module` d'origine et un interpréteur Python pour le reconstruire. Un `SavedModel` contient le graphe en Protocol Buffers.

34. JAX, XLA et TPU

Source : C3, p. 13 à 15

Avec des modèles à mille milliards de paramètres, le goulet passe de l'API à la synchronisation matérielle distribuée. PyTorch et TensorFlow gardent l'état (les poids) dans des objets et lancent des noyaux dynamiquement ; sur 10 000 GPU, cette gestion d'état devient un cauchemar. D'où l'idée : écrire des transformations mathématiques pures et laisser un compilateur les distribuer.

JAX (Google) n'est pas un framework de réseaux au sens classique : c'est un **compilateur d'algèbre linéaire** qui combine différentiation automatique et compilateur XLA.

- **Paradigme fonctionnel** : pas d'état mutable. Un réseau est une **fonction pure** (sortie dépendant seulement des entrées, aucun effet de bord). Pas de mise à jour en place : l'optimiseur prend anciens poids et gradients et renvoie de nouveaux poids immuables.
- **Pytree** : imbrication de dictionnaires et listes Python dont les feuilles sont des tableaux. JAX retire la structure, passe au compilateur uniquement les tableaux à plat, puis reconstruit l'arbre ; cette séparation structure et données permet une parallélisation automatique quasi sans surcoût.
- **Chaîne XLA** : `@jax.jit` trace le Python en langage intermédiaire **Jaxpr**, l'abaisse en **StableHLO** (jeu d'instructions indépendant du matériel), et le passe à **XLA** (Accelerated Linear Algebra), qui fusionne les opérations et produit du code machine (PTX pour NVIDIA, code TPU).

	GPU	TPU
Nature	SIMT généraliste, issu du rendu graphique, multiprocesseurs (SM)	ASIC (circuit spécifique) à grands réseaux systoliques d'unités MXU
Force	flexible : contrôle de flux dynamique, opérations non standard	débit et efficacité énergétique par watt sur les produits matriciels denses (LLM)
Coût	relit et réécrit les résultats intermédiaires en VRAM ou cache	programmabilité réduite, pas de contrôle de flux dynamique

Avantage systolique (schéma p. 15) : les poids sont fixés dans une grille physique, les entrées la traversent au rythme de l'horloge, et les résultats intermédiaires passent directement d'une ALU à sa voisine sans jamais toucher la mémoire globale.

35. La spirale classée par PyTorch

Source : C3, p. 15 à 17 ; NB3

L'exemple synthèse du cours 3 enchaîne génération, architecture, perte, optimiseur et boucle en cinq étapes.

```
X_train, y_train = generate_spiral_data() # tenseurs
device = torch.device("cuda" if torch.cuda.is_available()
                      else "cpu")
X_train, y_train = X_train.to(device), y_train.to(device)

class SpiralClassifier(nn.Module):
    def __init__(self, input_features=2, num_classes=3):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_features, 64), nn.ReLU(),
            nn.Linear(64, 64), nn.ReLU(),
            nn.Linear(64, num_classes)) # logits
    def forward(self, x):
        return self.network(x)
```

```

model = SpiralClassifier().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
for epoch in range(100):
    model.train()
    logits = model(X_train)           # A
    loss = criterion(logits, y_train)  # B
    optimizer.zero_grad()             # C
    loss.backward()                   # D
    optimizer.step()                  # E
model.eval()
with torch.no_grad():
    pred = model(X_train).argmax(dim=1)
    accuracy = (pred == y_train).float().mean()

```

- Les étiquettes sont en int64 (`np.int64`) : `CrossEntropyLoss` attend des indices de classe entiers longs (*rappel général*). Dans `SpiralDataset` du cours 2, elles étaient en uint8.
- `nn.Sequential` est l'équivalent PyTorch du `keras.Sequential`.
- Entraînement en *full batch* : les 3000 points passent à chaque époque, donc 100 mises à jour.
- La précision est mesurée sur les données d'entraînement (`X_train`) : elle ne dit rien de la généralisation. Un jeu de test séparé serait nécessaire (*rappel général*).

Paramètres (*rappel général*)

Linear(2, 64) : $128 + 64 = 192$; Linear(64, 64) : $4096 + 64 = 4160$;
 Linear(64, 3) : $192 + 3 = 195$. Total : 4547.

Piège

Le commentaire p. 16 parle de « zero-copy transition to VRAM if already pinned ». Un transfert CPU vers GPU copie les données dans la VRAM ; la mémoire « pinned » accélère ce transfert, elle ne le supprime pas (*rappel général*). Formulation du support **À VÉRIFIER**.

36. Préparer les labs

Source : L1 ; fiche CLAUDE.md ; C1, p. 1

Fonctionnement et rendu

Labs en ligne et asynchrones, sans séance. Un lien d'invitation Moodle crée un dépôt GitHub personnel par lab. Procédure obligatoire (L1, « Submitting assignments ») ; tout autre rendu n'est pas évalué :

1. travailler dans le dépôt créé par l'invitation ;
2. créer une branche `solution`, ne jamais travailler sur `main` ou `master` ; commits libres, penser à pousser ;
3. ouvrir une pull request intitulée `labX` (`lab1`, `lab2`...), source `solution`, cible la branche principale, assignée à l'enseignant de lab (nom GitHub sur Moodle ; pour le lab 1 : `balazskvancz`) ; la laisser ouverte ;
4. vérifier le contrôle automatique GitHub Actions (au minimum la présence de `neptun.txt`) ; corriger et repousser sur `solution` en cas d'erreur ;
5. questions : en commentaire de la PR, avec mention `@username`.

Si l'énoncé change, une PR des enseignants arrive dans le dépôt ; si `solution` existe déjà, changer sa cible vers `solution`, puis la fusionner.

Bonus : points pour des corrections non triviales des énoncés, par PR sur le dépôt `viaumsma001/laboratories` ; erreurs des supports de cours à signaler par issue.

Lab 1 : Python AI Environments and Simple Neural Networks

Installation imposée : `pip install numpy pandas matplotlib torch torchvision tensorflow` ; `scikit-learn` est utilisé sans figurer dans la commande. Notebook ou fichiers `.py` au choix. Validation : **8 points sur 20**. Captures PNG obligatoires, au nom exact, sinon rendu refusé.

Ex.	Contenu	Pts	Captures
1	NumPy (stats d'un tirage normal), Matplotlib (sin, cos sur $[0, 2\pi]$, titre, légende, grille), Pandas (vgsales.csv, category, groupby, filtre, top 3)	3	f1_1, f1_2, f1_3
2	régression linéaire PyTorch, $y = 2x + 1 + \epsilon$, $\sigma = 2$, split 60/20/20, courbes train et validation	4	f2_1, f2_2
3	classifieur softmax sur Iris, scaler, split 80/20, précision de test, courbe de perte	6	f3
4	classifieur MNIST avec DataLoaders, précision de test, courbe de perte	7	f4
	avis sur le lab	+1	aucune

Liens entre le lab 1 et le cours

Besoin du lab	Où c'est dans le cours
venv, pip, requirements	concept 9 (C1, p. 14 à 16)
np.linspace, randn, stats	concept 22 (C2, p. 24 à 26)
category, groupby, index	concepts 23 et 24 (C2, p. 28 à 31)
nn.Module, boucle en 5 étapes	concepts 29 et 30 (C3, p. 4 à 7)
CrossEntropyLoss sur des logits	concept 30 (C3, p. 17)

scikit-learn (introduit par L1, « not covered in detail during the lectures »): `train_test_split(X, y, test_size=0.2, random_state=42)`; `StandardScaler` avec `fit_transform` sur l'entraînement et seulement `transform` sur le test; `LinearRegression().fit` puis `predict`; métriques `mean_squared_error` et `r2_score`.

Piège

- Ajuster le scaler sur tout le jeu, ou refaire `fit` sur le test, fait fuir de l'information du test vers l'entraînement (*rappel général*).
- Exercice 2 : le texte demande 1000 valeurs, le squelette contient

`no_elements = 100`. Choix à expliciter dans le rendu. Découpage 60/20/20 de 1000 points : 600, 200, 200.

- Deadlines des labs **À VÉRIFIER** ; labs 2 à 6 : seuls les titres sont connus (NLP et embeddings ; LLM local ; RAG avec LangChain ; pipeline multimodal et outils de développement ; FastAPI et Docker).

Projet final. Les supports n'en disent qu'une phrase : « Final homework project must be prepared by the end of semester » (C1, p. 1). La fiche officielle compte 20 h de projet sur 120 h. Sujet, format, deadline et poids : **À VÉRIFIER**.

37. Méthode d'examen

Source : synthèse de ce document ; format du ZH **À VÉRIFIER**

Le format du ZH (QCM, questions ouvertes, lecture de code, durée, documents autorisés) n'est pas publié. Les supports étant rédigés comme un polycopié détaillé, préparer trois types de réponses.

À retenir

1. **Définir et comparer** : paradigmes, supervisé et non supervisé, vue et copie, SIMD et SIMT, CPU et GPU, eager et graphe statique, PyTorch et TensorFlow, GPU et TPU. Pour une comparaison, un tableau à 3 ou 4 critères.
2. **Calculer** : strides et offset, forme d'un broadcasting, valeurs de l'inspace, nombre de flottants par registre, paramètres d'une couche, batchs par époque, tailles d'un découpage.
3. **Lire du code** : aliasing de listes, MRO et `super()`, fermeture et `nonlocal`, décorateur, vue ou copie NumPy, ordre des 5 étapes d'entraînement.

Question	Méthode (numéro de concept)
strides d'un tableau contigu	taille d'élément à droite, puis produits vers la gauche (18)
offset de (i, j, k)	$\sum$ indice $\times$ stride (18)
deux formes compatibles ?	aligner à droite, compléter par 1, égal ou 1 (21)
vue ou copie ?	tranche : vue ; liste d'indices ou masque : copie 1D (19)
ordre de la MRO	enfant d'abord, parents dans l'ordre écrit, ancêtre commun à la fin (13)
que vaut a après $b = a$?	réaffectation : rien ; modification en place : tout (5)
paramètres d'un MLP	$\sum(\text{in} \times \text{out} + \text{out})$ (29)
mises à jour par époque	$[N/\text{batch}]$ (32)

Piège

- Répondre « Python est interprété » sans parler de bytecode ni de PVM.
- Dire que `arr[mask]` garde la forme 2D.
- Oublier `zero_grad()` ou le placer après `backward()`.
- Mettre un softmax avant `CrossEntropyLoss`.
- Confondre `__new__` (crée, reçoit cls) et `__init__` (remplit, reçoit self).
- Dire que `__x` rend un attribut réellement privé.

38. Aide-mémoire

À retenir	
Supervisé, non supervisé	$f(X; \theta) \approx P(Y X); P(X)$
Offset NumPy	$i s_0 + j s_1 + k s_2$
linspace	$x_i = \text{start} + i(\text{stop} - \text{start})/(\text{num} - 1)$
Loi normale standard	$p(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}, \mu = 0, \sigma = 1$
Z-score	$(X - \mu)/(\sigma + 10^{-8}), \text{axis}=0, \text{keepdims}=\text{True}$
ReLU	$\max(0, x)$
nn.Linear	$y = xA^T + b$; in $\times$ out + out paramètres
Sigmoïde, softmax	$1/(1 + e^{-z}); e^{z_i} / \sum_j e^{z_j}$ (rappel général)
Descente de gradient	$\theta \leftarrow \theta - \eta \nabla_{\theta} L$ (rappel général)
Flottants par registre	bits / 32 : SSE et NEON 4, AVX 8, AVX-512 16
Spirale	$r = 5 i/n, \theta = 4 i/n + 4c + \mathcal{N}(0, \text{noise})$

Python	
Exécution	source, AST, bytecode (.pyc), PVM (ceval.c)
Mémoire	étiquettes sur objets; comptage de références et GC; GIL
Typage	dynamique et fort; int arbitraire; bool sous-classe de int
Coûts de liste	accès, append, pop() $O(1)$; insert(0), pop(0), remove, in $O(N)$
Visibilité	x public, _x convention, __x devient _Classe__x
Objets	__new__(cls) crée, __init__(self) remplit, __call__ rend callable
MRO	C3 : ordre cohérent, préséance locale, monotonie; super() = suivant dans la MRO
Arguments	*args tuple, **kwargs dict
Jupyter	Shell, IOPub, Stdin, Control, Heartbeat sur ZeroMQ

NumPy, Pandas	
Vue	tranches, .T, reshape : $O(1)$
Copie	liste d'indices, masque (résultat 1D) : $O(N)$
Broadcasting	aligner à droite, compléter par 1, égal ou 1, stride 0
Pandas	fillna, astype("category"), drop(columns=...), jointure par index

Frameworks	
PyTorch	allocateur à cache ; autograd inverse, DAG dynamique ; nn.Module et __setattr__
5 étapes	forward, loss, zero_grad, backward, step
PyTorch 2	torch.compile : TorchDynamo, TorchInductor, Triton
TensorFlow	graphe statique GraphDef ; TF2 eager + Keras ; compile, fit ; XLA
JAX	fonctions pures, pytrees, jit : Jaxpr, StableHLO, XLA
TPU	ASIC, réseau systolique de MXU, poids fixes
